AutoIA V1 Test and Validation Strategy

Release: AutoIA `1.0.2`
Purpose: Explain what is tested, what each evidence layer means, and what is not implied by passing tests.

1. Validation philosophy

AutoIA makes machine-rigorous enclosure claims. A large test count is therefore not itself the proof. The proof basis is the documented validated arithmetic, outward rounding, domain rules, representation semantics, and fail-closed execution contract. Tests attack implementation mistakes and verify that the software conforms to those rules over broad and adversarial cases.

The release evidence is intentionally layered:

1. **Named regression tests** exercise stable public and internal contracts.
2. **Pytest subtests** provide many additional parameterized numerical witnesses.
3. **Exact-rational input audit** independently checks decimal/rational endpoint containment without trusting floating-point comparisons.
4. **High-precision GP posterior audit** independently checks the transparent GP posterior enclosure against 80-digit reference evaluation.
5. **Numbered examples** exercise ordinary user-facing source paths.
6. **Clean-wheel execution** repeats user-facing behavior from an isolated wheel install rather than the source checkout.
7. **Release-content audit** verifies the package boundary across source, wheel, PDFs, and release archives.

2. Final 1.0.2 counts

The final source collection contains **388 named pytest cases in 59 test files**. The build host does not have the optional GPyTorch and BoTorch packages, so the exact result is:

Evidence	Result
Named pytest cases collected	388
Named pytest cases passed	386
Optional live-GP cases skipped	2
Named pytest failures	0
Pytest subtests passed	2,405
Exact-rational input-containment checks	10,081
80-digit GP posterior containment checks	504
Numbered example scripts	29
Numbered scripts exiting successfully	29
Optional-dependency example skips	2

The two optional skips are not substituted by a claim of live third-party execution. They are complemented by dependency-free structural semantic tests and complete runnable live examples that execute automatically when the optional packages are present.

3. Trusted arithmetic and elementary semantics

The regression suite checks scalar interval construction, outward-rounded arithmetic, elementary functions, special functions, exact powers, normal PDF/CDF functions, array behavior, reductions, domain proofs, and typed failure semantics. Boundary tests target overflow/underflow-sensitive behavior, zero-crossing division, closed endpoints, exact constants, and operations whose mathematical domains must be proved before evaluation.

4. Representation evidence

Tests cover the plain interval, shared-generator, and polynomial enclosure layers; packed polynomial ranging; Bernstein conversion/ranging; monotonicity ranging; controller selection; cancellation preservation; subdivision; and conversion boundaries. These tests check both enclosure correctness and the metadata/proof-state invariants needed to keep proposal decisions separate from trusted proof quantities.

5. Ordinary-source semantic execution

Tests exercise local semantic dispatch rather than requiring user code to import interval-specific replacements. Covered public-style semantics include NumPy and `math` operations, arrays, branching, `numpy.linalg.solve`, selected SciPy functions, `scipy.optimize.root_scalar`, `scipy.optimize.fsolve`, and fixed-data linear `RegularGridInterpolator`.

Recognized unsupported operations are tested to fail explicitly. AutoIA must not silently run unsupported uncertain arithmetic in ordinary floating point and then mark the result rigorous.

6. Prepared and many-box execution

The suite covers `prepare(...)`, repeated finite boxes, compact and streaming many-box routes, serial/process agreement, conservative automatic CPU scheduling, and trusted-result metadata. Accelerator proposal helpers are tested as untrusted proposal computations; they do not enter the proof base.

7. Matrix-family evidence

SPD matrix-family tests cover validated triangular operations and structured matrix semantics. An independent high-precision family audit remains separate

from the stable named-test count so that the implementation is checked against a numerically independent reference route.

8. Input-specification audit

AutoIA 1.0.2 retains the 1.0.1 input contract:

- `Interval(lo, hi)` means the exact stored binary64 endpoints supplied by the Python values;
- `Interval.enclosing(...)` encloses intended exact decimal/rational endpoints;
- `Interval.outward(...)` explicitly adds one-ULP outward padding to stored floats and does not claim arbitrary provenance.

`tools/audit_input_specification.py` performs **10,081 exact-rational checks**. The audit converts proof endpoints to exact rational values and verifies set containment without using binary64 comparison as the reference truth.

9. Transparent fitted-GP evidence

Version 1.0.2 adds transparent semantic recognition for a narrow fitted scalar exact-GP subset. Dependency-free regression tests use structural model doubles to exercise:

- ordinary `torch.tensor`-style query construction;
- an ordinary fitted-model call captured as a global;
- a fitted-model call captured in a Python closure;
- RBF and Matern posterior semantics;
- the lower-level expert freeze API;
- explicit fail-closed rejection of an unsupported kernel.

Two additional tests use real GPyTorch and BoTorch models when those optional packages are installed. They are skipped explicitly on the build host because those packages are unavailable.

The independent high-precision audit evaluates four frozen kernel profiles: RBF and Matern `nu=1/2`, `3/2`, and `5/2`. For each profile it checks posterior mean and variance containment at 63 high-precision query points, for

```
4 profiles * 63 points * 2 quantities = 504 containment checks.
```

The reference uses 80-digit arithmetic while interpreting all frozen binary64 parameters as their exact stored values. This audit is conditional on the frozen fitted state; it does not certify statistical fitting, autograd, or training convergence.

10. Numbered examples

The release contains numbered examples `00` through `28`. They cover the normal API progressively, ending with complete ordinary GPyTorch and BoTorch fitting examples. All 29 scripts exit successfully in the source and clean-wheel release runs. On the build host examples 27 and 28 print explicit dependency skips rather than pretending that unavailable third-party packages were exercised.

11. Release-content policy

A regression scanner and final packaging audit enforce the package boundary. The prohibited downstream-package token must have zero case-insensitive occurrences in distributed file names or text. The packaging pass repeats the scan on:

- the cleaned source tree;
- the extracted source ZIP;
- the extracted wheel;
- text extracted from every distributed PDF;
- the extracted complete-release ZIP.

The final release record reports these scans only after the actual artifacts have been built and checked.

12. What the evidence does not prove

The test suite does not imply that arbitrary Python, arbitrary Torch code, arbitrary kernels, arbitrary transforms, or arbitrary third-party libraries are certified. AutoIA recognizes a documented subset and fails closed outside it. Likewise, fitted GP hyperparameter estimation and training arithmetic are not certified. The machine-rigorous claim concerns supported validated execution conditional on the documented stored inputs and fixed fitted numerical state.

13. Release acceptance criterion

AutoIA 1.0.2 is accepted only if all of the following hold simultaneously:

- 386 named cases pass, 2 optional live-GP cases skip for missing dependencies, and no named case fails;
- all 2,405 pytest subtests pass;
- all 10,081 exact-input checks pass;
- all 504 high-precision GP posterior checks pass;
- all 29 numbered scripts exit successfully in source and clean-wheel runs;
- installed wheel code matches the source package code;
- all prohibited-name scans return zero;
- all five synchronized PDFs render without clipping or broken glyphs;
- wheel, source ZIP, complete-release ZIP, release notes, verification record, documentation, and SHA-256 manifest are mutually consistent.